

Aula 12 - Upload de Arquivos e Paginação

Duração: 4 horas (1h teórica + 3h prática)

Parte Teórica (1 hora)

1. Revisão da Aula Anterior (15 min)

- Discussão sobre:
 - Fluxo de autenticação JWT.
 - Dúvidas sobre interceptors e guards no Angular.

2. Upload de Arquivos no Spring Boot (25 min)

Métodos de Armazenamento

Tipo	Vantagens	Desvantagens
Banco de Dados	ACID, transações	Performance ruim para grandes arquivos
Sistema de Arquivos	Simplicidade	Escalabilidade limitada
Cloud (S3/Azure)	Escalável, redundante	Custo e complexidade

Anotações Principais

- `@RequestParam("file") MultipartFile file` : Recebe o arquivo no controller.
- `Files.copy()` : Salva o arquivo no sistema de arquivos.

3. Paginação (20 min)

Spring Data JPA Pagination

- Interface `Page` :

```
java
    Page<Product> findAll(Pageable pageable);
```

- Parâmetros de URL:

- `?page=0&size=10&sort=name,asc`

Angular Material Paginator

- Componente pronto para paginação em tabelas.

Parte Prática (3 horas)

Atividade: Sistema de Upload com Paginação

Passo 1: Backend - Upload de Arquivos (1h)

1. Configuração (application.properties):

```
properties

# Tamanho máximo do arquivo (10MB)
spring.servlet.multipart.max-file-size=10MB
spring.servlet.multipart.max-request-size=10MB
```

2. Controller (FileController.java):

```
java

@RestController
@RequestMapping("/api/files")
public class FileController {
    private final Path root = Paths.get("uploads");

    @PostMapping
    public ResponseEntity<String> upload(@RequestParam("file") MultipartFile file) {
        try {
            if (!Files.exists(root)) Files.createDirectory(root);
            String filename = UUID.randomUUID() + "_" + file.getOriginalFilename();

            Files.copy(file.getInputStream(), root.resolve(filename));
            return ResponseEntity.ok(filename);
        } catch (IOException e) {
            throw new RuntimeException("Falha no upload: " + e.getMessage());
        }
    }
}
```

3. Habilitar acesso a arquivos estáticos:

```
java

@Configuration
public class WebConfig implements WebMvcConfigurer {
```

```

@Override
public void addResourceHandlers(ResourceHandlerRegistry registry) {
    registry.addResourceHandler("/uploads/**")
        .addResourceLocations("file:uploads/");
}
}

```

Passo 2: Frontend - Upload com Angular (1h)

1. Serviço (file.service.ts):

```

typescript

@Injectable({ providedIn: 'root' })
export class FileService {
    private apiUrl = 'http://localhost:8080/api/files';

    constructor(private http: HttpClient) {}

    upload(file: File) {
        const formData = new FormData();
        formData.append('file', file);
        return this.http.post<string>(this.apiUrl, formData);
    }
}

```

2. Componente (upload.component.ts):

```

typescript

export class UploadComponent {
    selectedFile?: File;

    onFileSelected(event: any) {
        this.selectedFile = event.target.files[0];
    }

    upload() {
        if (this.selectedFile) {
            this.fileService.upload(this.selectedFile).subscribe({
                next: (filename) => alert(`Arquivo ${filename} salvo!`),
                error: (err) => alert('Erro no upload!')
            });
        }
    }
}

```

3. Template (upload.component.html):

```

html

```

```
<input type="file" (change)="onFileSelected($event)">
<button (click)="upload()">Enviar</button>
```

Passo 3: Paginação no Backend (30 min)

1. Repository (ProductRepository.java):

```
java

public interface ProductRepository extends JpaRepository<Product, Long> {
    Page<Product> findAll(Pageable pageable);
}
```

2. Controller (ProductController.java):

```
java

@GetMapping
public Page<Product> getAll(
    @RequestParam(defaultValue = "0") int page,
    @RequestParam(defaultValue = "10") int size
) {
    return repository.findAll(PageRequest.of(page, size));
}
```

Passo 4: Paginação no Frontend (30 min)

1. Instalar Angular Material:

```
bash

ng add @angular/material
```

2. Componente com Paginação (product-list.component.html):

```
html

<table mat-table [dataSource]="dataSource">
  <!-- Colunas -->
</table>
<mat-paginator [pageSizeOptions]="[5, 10, 20]"></mat-paginator>
```

3. Lógica (product-list.component.ts):

```
typescript

dataSource = new MatTableDataSource<Product>();
@ViewChild(MatPaginator) paginator!: MatPaginator;

ngOnInit() {
    this.productService.getAll(0, 10).subscribe(products => {
```

```
        this.dataSource.data = products.content;
        this.dataSource.paginator = this.paginator;
    });
}
```

Tarefa para Casa

1. Validação de Arquivos:

- Restringir para apenas imagens (JPEG/PNG).

2. Paginação Avançada:

- Implementar filtros por nome/categoria.

Próxima Aula

- **Tópico:** Testes Unitários (JUnit 5 + Mockito).
- **Atividade prática:** Testar serviços e controllers.

Resultado Esperado

- Sistema completo com:
 - Upload de arquivos para o servidor.
 - Listagem paginada de produtos.

plaintext

- ✓ Upload funcional
- ✓ Paginação backend + frontend
- ✓ Pronto para testes automatizados

<https://www.baeldung.com/wp-content/uploads/2021/11/spring-rest-upload-sequence.png> (Fluxo de upload no Spring) 🚀

